

JAVA-

Q1. What is garbage collection in java?

When a program runs, it uses memory to store data (like variables and objects). Over time, some of that data is no longer needed—but it still sits in memory.

Garbage collection (GC) is the process of:

- ☞ finding that unused data (“garbage”)
- ☞ and freeing up the memory so it can be reused

Q2. What is dynamic binding in java?

Dynamic binding (also called **late binding** or **runtime polymorphism**) in Java means:

The method that gets executed is decided at **runtime**, not at compile time.

It mainly happens when:

- **method overriding** is used
- a **parent class reference** points to a **child class object**

✓ Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {

        Animal a = new Dog();    // parent reference, child object
        a.sound();                // Which sound() will run?
    }
}
```

◆ Output

Dog barks

Q3. Static Binding vs Dynamic Binding

Static Binding	Dynamic Binding
Compile-time	Runtime
Method call fixed early	Method call decided later
Used in method overloading	Used in method overriding
Faster	Slightly slower

Methods that use Static Binding -

These are resolved at compile time:

- `static methods`
- `private methods`
- `final methods`

Because they cannot be overridden.

Q4. What are Strings in java?

In Java, a **String** is an object used to store a sequence of characters (text).

Example:

```
String name = "Sudhanshu";
```

Here, "Sudhanshu" is a String.

Java provides the `String` class inside:

```
java.lang.String
```

Creating Strings-

1. Using String Literal

```
String s1 = "Hello";
```

Stored in the **String Constant Pool**.

2. Using `new` Keyword

```
String s2 = new String("Hello");
```

Creates a new object in heap memory.

Q5. String are immutable?

Strings are Immutable

Immutable means:

Once a String object is created, its value cannot be changed.

Example:

```
String s = "Hello";  
s.concat(" World");  
  
System.out.println(s);
```

Output

Hello

Because `concat()` creates a new String instead of modifying the old one.

Correct way:

```
s = s.concat(" World");
```

Q6. What is String Constant Pool (SCP)?

Java stores String literals in a special memory area called:

String Constant Pool

Example:

```
String a = "Java";  
String b = "Java";
```

Both `a` and `b` point to the same object to save memory.

Q7. Common String Methods?

Method	Purpose
<code>length()</code>	Returns length
<code>charAt()</code>	Gets character
<code>substring()</code>	Extracts part
<code>equals()</code>	Compares content
<code>equalsIgnoreCase()</code>	Ignores case
<code>toUpperCase()</code>	Uppercase
<code>toLowerCase()</code>	Lowercase
<code>trim()</code>	Removes spaces
<code>contains()</code>	Checks text
<code>replace()</code>	Replaces characters

Q8. String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	✗ No	✓ Yes	✓ Yes
Thread Safe	✓ Yes	✗ No	✓ Yes
Performance	Slower for modifications	Fast	Moderate

Use:

- `String` → fixed text
- `StringBuilder` → frequent changes
- `StringBuffer` → thread-safe modifications

Q9. String Comparison?

1. Using `==`

Compares memory addresses.

```
String a = "Java";  
String b = "Java";  
  
System.out.println(a == b);
```

Output:

```
true
```

2. Using `equals()`

Compares actual content.

```
String a = new String("Java");  
String b = new String("Java");  
  
System.out.println(a.equals(b));
```

Output:

```
True
```

```
String a = new String("Java");  
String b = "Java";  
  
System.out.println(a == b);
```

✓ **Output**

```
false
```

Q10. What are ways of creating thread in java?

In Java, there are **mainly 2 ways** to create a thread:

1. **By extending Thread class**
2. **By implementing Runnable interface**

1. Extending Thread Class

You create a class that extends `Thread` and override the `run()` method.

Example

```
class MyThread extends Thread {  
  
    @Override  
    public void run() {  
        System.out.println("Thread is running...");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        MyThread t1 = new MyThread();  
  
        t1.start();    // starts new thread  
    }  
}
```

Important

- `run()` contains the task
- `start()` creates a new thread internally and calls `run()`

If you call:

```
t1.run();
```

it behaves like a normal method call, not a separate thread.

✓ 2. Implementing `Runnable` Interface

This is the **preferred approach**.

Example

```
class MyRunnable implements Runnable {

    @Override
    public void run() {
        System.out.println("Runnable thread running...");
    }
}

public class Main {
    public static void main(String[] args) {

        MyRunnable obj = new MyRunnable();

        Thread t1 = new Thread(obj);

        t1.start();
    }
}
```

✓ Why `Runnable` is Preferred?

Because Java supports:

- single inheritance only

If you extend `Thread`, you cannot extend another class.

But with `Runnable`, your class can still extend another class.

3. Using Lambda Expression

Because `Runnable` is a functional interface.

```
public class Main {
    public static void main(String[] args) {

        Runnable r = () -> {
            System.out.println("Thread using lambda");
        };

        Thread t = new Thread(r);

        t.start();
    }
}
```

Q11. Difference Between Thread and Runnable

Thread Class	Runnable Interface
Extends <code>Thread</code>	Implements <code>Runnable</code>
Cannot extend another class	Can extend another class
Less flexible	More flexible
Tightly coupled	Better design

Q12. Thread Lifecycle and Common Thread Methods

--NEW → RUNNABLE → RUNNING → TERMINATED

Method	Purpose
<code>start()</code>	Starts thread
<code>run()</code>	Task logic
<code>sleep()</code>	Pause thread
<code>join()</code>	Wait for another thread
<code>isAlive()</code>	Checks if running

Q13. What are collections in java?

In Java, **Collections** are a framework used to store, manage, and manipulate groups of objects dynamically.

Main Interfaces in Collections -

Interface	Description
<code>List</code>	Ordered, duplicates allowed
<code>Set</code>	Unique elements
<code>Queue</code>	FIFO structure
<code>Map</code>	Key-value pairs

1. List Interface

Maintains insertion order and allows duplicates.

Common classes:

- ArrayList
- LinkedList
- Vector

Example

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        List<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("Java");

        System.out.println(list);
    }
}
```

Output

```
[Java, Python, Java]
```

2. Set Interface

Stores unique elements only.

Common classes:

- HashSet
- LinkedHashSet
- TreeSet

Example

```
Set<Integer> set = new HashSet<>();

set.add(10);
set.add(20);
set.add(10);
```

```
System.out.println(set);
```

Output

```
[10, 20]
```

Duplicate 10 removed automatically.

3. Queue Interface

Follows FIFO (First In First Out).

Common classes:

- PriorityQueue
- LinkedList

Example

```
Queue<Integer> q = new LinkedList<>();
```

```
q.add(1);  
q.add(2);
```

```
System.out.println(q.poll());
```

Output

```
1
```

4. Map Interface

Stores data as:

```
key -> value
```

Common classes:

- HashMap
- LinkedHashMap
- TreeMap

Example

```
Map<Integer, String> map = new HashMap<>();

map.put(1, "Java");
map.put(2, "Python");

System.out.println(map);
```

Output

```
{1=Java, 2=Python}
```

Q13. Common Collection Classes

Class	특징
ArrayList	Fast random access
LinkedList	Fast insertion/deletion
HashSet	Unique unordered data
TreeSet	Sorted unique data
HashMap	Key-value storage
PriorityQueue	Priority-based queue

Advantages of Collections

- Dynamic size
- Built-in algorithms
- Easy sorting/searching
- Reduces coding effort
- Generic support

✓ Generics in Collections

```
List<String> list = new ArrayList<>();
```

String means:

only String objects allowed

Provides:

- type safety
- compile-time checking

Important Utility Class

`Collections` class provides helper methods:

```
Collections.sort(list);  
Collections.reverse(list);  
Collections.shuffle(list);
```

Q14. What is bucketing in HashMap?

In Java, **bucketing in HashMap** refers to the way data is stored internally using **buckets**.

A bucket is basically:

a location in an internal array where key-value pairs are stored.

Q15. What is ConcurrentHashMap?

`HashMap` and `ConcurrentHashMap` are both used to store key-value pairs in Java, but the major difference is:

- ✓ `HashMap` is **not thread-safe**
- ✓ `ConcurrentHashMap` is **thread-safe**

Basic Difference

Feature	HashMap	ConcurrentHashMap
Thread-safe	✗ No	✓ Yes
Synchronization	None	Internal synchronization
Performance	Faster in single thread	Better for multi-threading
Null Keys	✓ One null key allowed	✗ Not allowed
Null Values	✓ Multiple allowed	✗ Not allowed
Iterator	Fail-fast	Fail-safe
Package	<code>java.util</code>	<code>java.util.concurrent</code>

1. HashMap

Used in:

- single-threaded applications
- when synchronization is not needed

Example

```
HashMap<Integer, String> map = new HashMap<>();  
  
map.put(1, "Java");  
map.put(2, "Python");
```

⚠ Problem with HashMap in Multithreading

If multiple threads modify `HashMap` simultaneously:

- data inconsistency may happen
- infinite loops may occur (older Java versions)
- corruption possible

2. ConcurrentHashMap

Designed for:

concurrent/multi-threaded environments

Example

```
ConcurrentHashMap<Integer, String> map =  
    new ConcurrentHashMap<>();  
  
map.put(1, "Java");  
map.put(2, "Python");
```

🔍 How ConcurrentHashMap Works

Older Java:

- segmented locking

Java 8+:

- bucket-level locking + CAS operations

This means:

multiple threads can work simultaneously on different buckets

So performance is much better than full synchronization.

Null Handling Difference

HashMap

```
map.put(null, "A");  
map.put(1, null);
```

✓ Allowed

ConcurrentHashMap

```
map.put(null, "A");
```

✗ Throws:

NullPointerException

Iterator Difference

HashMap → Fail-Fast

If map changes during iteration:

ConcurrentModificationException

ConcurrentHashMap → Fail-Safe

Allows modification during iteration safely.

✓ Example of Thread Safety

```
Map<Integer, String> map =  
    new ConcurrentHashMap<>();  
  
Runnable task = () -> {  
    for(int i = 0; i < 1000; i++) {  
        map.put(i, "Value");  
    }  
};  
  
new Thread(task).start();  
new Thread(task).start();
```

Safe for concurrent access.